

```
!pip install -q pandas numpy matplotlib seaborn scipy scikit-learn plotly
```

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from pathlib import Path
from scipy.stats import ks_2samp, chi2_contingency
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    roc_auc_score
)

RANDOM_STATE = 42
rng = np.random.default_rng(RANDOM_STATE)

pd.set_option("display.max_columns", None)

print("Libraries imported successfully.")
```

Libraries imported successfully.

```
PROJECT_DIR = Path("/content/AIF-8B-AI-Model-Drift-Investigation")

DATA_DIR = PROJECT_DIR / "data"
REPORT_DIR = PROJECT_DIR / "reports"
VIS_DIR = PROJECT_DIR / "visualisations"
DOCS_DIR = PROJECT_DIR / "docs"

for folder in [
    DATA_DIR,
    REPORT_DIR,
    VIS_DIR,
    DOCS_DIR
]:
    folder.mkdir(parents=True, exist_ok=True)

print("Project folders created:")
print(PROJECT_DIR)
```

Project folders created:
/content/AIF-8B-AI-Model-Drift-Investigation

```
# Number of simulated records
n_records = 50_000

# Create monthly dates over 18 months
dates = pd.date_range(
    start="2024-01-01",
    periods=18,
    freq="MS"
)

months = rng.choice(
    dates,
    size=n_records,
    replace=True
)

months = pd.DatetimeIndex(pd.to_datetime(months))

month_index = (
    (months.year - dates[0].year) * 12
    + (months.month - dates[0].month)
)

# Simulated customer and transaction features
transaction_amount = rng.lognormal(
    mean=7.0,
    sigma=0.8,
    size=n_records
)

transaction_amount = transaction_amount.clip(10, 200_000)
```

```
transaction_frequency = rng.poisson(
    lam=5,
    size=n_records
)

account_age_days = rng.integers(
    low=1,
    high=3_000,
    size=n_records
)

customer_age = rng.integers(
    low=18,
    high=80,
    size=n_records
)

device_risk_score = rng.beta(
    a=2,
    b=5,
    size=n_records
)

location_risk_score = rng.beta(
    a=2,
    b=4,
    size=n_records
)

merchant_risk_score = rng.beta(
    a=2,
    b=5,
    size=n_records
)

failed_login_count = rng.poisson(
    lam=1.5,
    size=n_records
)

beneficiary_age_days = rng.integers(
    low=1,
    high=2_000,
    size=n_records
)

is_new_device = rng.binomial(
    n=1,
    p=0.20,
    size=n_records
)

is_new_beneficiary = rng.binomial(
    n=1,
    p=0.15,
    size=n_records
)

hour_of_day = rng.integers(
    low=0,
    high=24,
    size=n_records
)

merchant_category = rng.choice(
    [
        "Retail",
        "Utilities",
        "Travel",
        "Gaming",
        "Healthcare",
        "Education"
    ],
    size=n_records
)

channel = rng.choice(
    [
        "Mobile App",
        "Web",
        "ATM",
        "POS"
```

```

    ],
    size=n_records,
    p=[0.60, 0.20, 0.10, 0.10]
)

customer_segment = rng.choice(
    [
        "New",
        "Regular",
        "Premium"
    ],
    size=n_records,
    p=[0.25, 0.60, 0.15]
)

# Fraud probability changes after month 9.
# This simulates concept drift.
drift_effect = np.where(
    month_index >= 9,
    0.20,
    0.00
)

fraud_score = (
    -3.0
    + 0.000012 * transaction_amount
    + 0.60 * device_risk_score
    + 0.70 * location_risk_score
    + 0.80 * merchant_risk_score
    + 0.25 * is_new_device
    + 0.30 * is_new_beneficiary
    + 0.15 * failed_login_count
    + drift_effect
)

fraud_probability = 1 / (
    1 + np.exp(-fraud_score)
)

actual_fraud = rng.binomial(
    n=1,
    p=fraud_probability
)

# Model predictions become less effective after month 9.
model_score = (
    fraud_probability
    + rng.normal(0, 0.08, n_records)
)

model_score = np.clip(
    model_score,
    0,
    1
)

predicted_fraud = (
    model_score >= 0.50
).astype(int)

# Model performance labels
true_positive = (
    (predicted_fraud == 1)
    & (actual_fraud == 1)
).astype(int)

false_positive = (
    (predicted_fraud == 1)
    & (actual_fraud == 0)
).astype(int)

true_negative = (
    (predicted_fraud == 0)
    & (actual_fraud == 0)
).astype(int)

false_negative = (
    (predicted_fraud == 0)
    & (actual_fraud == 1)
).astype(int)

# Dataset 1: Performance log

```

```

performance_log = pd.DataFrame({
    "date": months,
    "month_index": month_index,
    "total_transactions": 1,
    "actual_fraud": actual_fraud,
    "predicted_fraud": predicted_fraud,
    "true_positive": true_positive,
    "false_positive": false_positive,
    "true_negative": true_negative,
    "false_negative": false_negative
})

# Dataset 2: Feature distributions
feature_distributions = pd.DataFrame({
    "date": months,
    "month_index": month_index,
    "transaction_amount": transaction_amount,
    "transaction_frequency": transaction_frequency,
    "account_age_days": account_age_days,
    "customer_age": customer_age,
    "device_risk_score": device_risk_score,
    "location_risk_score": location_risk_score,
    "merchant_risk_score": merchant_risk_score,
    "failed_login_count": failed_login_count,
    "beneficiary_age_days": beneficiary_age_days,
    "is_new_device": is_new_device,
    "is_new_beneficiary": is_new_beneficiary,
    "hour_of_day": hour_of_day,
    "merchant_category": merchant_category,
    "channel": channel,
    "customer_segment": customer_segment
})

# Dataset 3: Sample predictions
sample_predictions = pd.DataFrame({
    "date": months,
    "month_index": month_index,
    "model_score": model_score,
    "predicted_fraud": predicted_fraud,
    "actual_fraud": actual_fraud
})

# Dataset 4: External events
external_events = pd.DataFrame({
    "date": dates,
    "month_index": range(len(dates)),
    "event_type": [
        "Normal operations",
        "Normal operations",
        "Normal operations",
        "Normal operations",
        "Normal operations",
        "Normal operations",
        "Normal operations",
        "UPI adoption increase",
        "New fraud pattern observed",
        "New fraud pattern observed",
        "Security policy change",
        "New beneficiary cooling period",
        "Digital arrest scam increase",
        "QR manipulation increase",
        "SIM swap fraud increase",
        "Monitoring threshold review",
        "Model review initiated"
    ],
    "severity": [
        "Low",
        "Low",
        "Low",
        "Low",
        "Low",
        "Low",
        "Low",
        "Low",
        "Low",
        "Medium",
        "High",
        "High",
        "Medium",
        "Medium",
        "High",
        "High"
    ]
})

```

```

        "High",
        "Medium",
        "High"
    ]
})

# Save all datasets
performance_log.to_csv(
    DATA_DIR / "performance_log.csv",
    index=False
)

feature_distributions.to_csv(
    DATA_DIR / "feature_distributions.csv",
    index=False
)

sample_predictions.to_csv(
    DATA_DIR / "sample_predictions.csv",
    index=False
)

external_events.to_csv(
    DATA_DIR / "external_events.csv",
    index=False
)

print("All four datasets created successfully.")

print("Performance log:", performance_log.shape)
print("Feature distributions:", feature_distributions.shape)
print("Sample predictions:", sample_predictions.shape)
print("External events:", external_events.shape)

```

All four datasets created successfully.
 Performance log: (50000, 9)
 Feature distributions: (50000, 17)
 Sample predictions: (50000, 5)
 External events: (18, 4)

```

datasets = {
    "performance_log": performance_log,
    "feature_distributions": feature_distributions,
    "sample_predictions": sample_predictions,
    "external_events": external_events
}

for name, data in datasets.items():
    print("\n" + "=" * 60)
    print(name)
    print("=" * 60)

    print("Rows and columns:", data.shape)
    print("Missing values:", data.isnull().sum().sum())
    print("Duplicate rows:", data.duplicated().sum())
    print("Date range:", data["date"].min(), "to", data["date"].max())
    print(data.head(3))

```

```

1      0      0      1      0
2      0      0      1      0

```

```

=====
feature_distributions
=====
Rows and columns: (50000, 17)
Missing values: 0
Duplicate rows: 0
Date range: 2024-01-01 00:00:00 to 2025-06-01 00:00:00
   date month_index transaction_amount transaction_frequency \
0 2025-02-01         13         391.092201                 6

```

```

1      0      0      2      Utilities
2      1      0      5      Utilities

      channel customer_segment
0 Mobile App      Premium
1 Mobile App      Premium
2 Mobile App      Regular

=====
sample_predictions
=====
Rows and columns: (50000, 5)
Missing values: 0
Duplicate rows: 3202
Date range: 2024-01-01 00:00:00 to 2025-06-01 00:00:00
      date month_index model_score predicted_fraud actual_fraud
0 2025-02-01      13      0.038600      0      0
1 2025-04-01      15      0.109725      0      0
2 2024-11-01      10      0.129085      0      0

=====
external_events
=====
Rows and columns: (18, 4)
Missing values: 0
Duplicate rows: 0
Date range: 2024-01-01 00:00:00 to 2025-06-01 00:00:00
      date month_index event_type severity
0 2024-01-01      0 Normal operations Low
1 2024-02-01      1 Normal operations Low
2 2024-03-01      2 Normal operations Low

```

```

import os

for root, folders, files_in_folder in os.walk(PROJECT_DIR):
    level = root.replace(str(PROJECT_DIR), "").count(os.sep)
    indentation = " " * level
    print(f"{indentation}{os.path.basename(root)}")

    for filename in files_in_folder:
        print(f"{indentation} {filename}")

```

```

AIF-8B-AI-Model-Drift-Investigation/
docs/
visualisations/
reports/
data/
    feature_distributions.csv
    sample_predictions.csv
    external_events.csv
    performance_log.csv

```

```

# Load the generated datasets

performance_log = pd.read_csv(
    DATA_DIR / "performance_log.csv",
    parse_dates=["date"]
)

feature_distributions = pd.read_csv(
    DATA_DIR / "feature_distributions.csv",
    parse_dates=["date"]
)

sample_predictions = pd.read_csv(
    DATA_DIR / "sample_predictions.csv",
    parse_dates=["date"]
)

external_events = pd.read_csv(
    DATA_DIR / "external_events.csv",
    parse_dates=["date"]
)

print("Datasets loaded successfully.")

print("Performance log:", performance_log.shape)
print("Feature distributions:", feature_distributions.shape)
print("Sample predictions:", sample_predictions.shape)
print("External events:", external_events.shape)

```

```

Datasets loaded successfully.
Performance log: (50000, 9)
Feature distributions: (50000, 17)
Sample predictions: (50000, 5)
External events: (18, 4)

```

```
def data_quality_report(data, dataset_name):
    """
    Generate a basic data-quality report for a dataset.

    Parameters
    -----
    data : pandas.DataFrame
        Dataset to inspect.

    dataset_name : str
        Name of the dataset.
    """

    print("=" * 60)
    print(dataset_name)
    print("=" * 60)

    print("Number of rows:", data.shape[0])
    print("Number of columns:", data.shape[1])
    print("Missing values:", data.isnull().sum().sum())
    print("Duplicate rows:", data.duplicated().sum())

    print("\nMissing values by column:")
    display(data.isnull().sum().to_frame("missing_values"))

    print("\nData types:")
    display(data.dtypes.to_frame("data_type"))

    print()

    data_quality_report(
        performance_log,
        "Performance Log"
    )

    data_quality_report(
        feature_distributions,
        "Feature Distributions"
    )

    data_quality_report(
        sample_predictions,
        "Sample Predictions"
    )

    data_quality_report(
        external_events,
        "External Events"
    )
```



```
=====
Performance Log
=====
Number of rows: 50000
Number of columns: 9
Missing values: 0
Duplicate rows: 49960
```

Missing values by column:

	missing_values	
date	0	
month_index	0	
total_transactions	0	
actual_fraud	0	
predicted_fraud	0	
true_positive	0	
false_positive	0	
true_negative	0	
false_negative	0	

Data types:

	data_type
date	datetime64[ns]
month_index	int64
total_transactions	int64
actual_fraud	int64
predicted_fraud	int64
true_positive	int64
false_positive	int64
true_negative	int64
false_negative	int64

```
=====
Feature Distributions
=====
Number of rows: 50000
Number of columns: 17
Missing values: 0
Duplicate rows: 0
```

Missing values by column:

	missing_values
date	0
month_index	0
transaction_amount	0
transaction_frequency	0
account_age_days	0
customer_age	0
device_risk_score	0
location_risk_score	0
merchant_risk_score	0
failed_login_count	0
beneficiary_age_days	0
is_new_device	0
is_new_beneficiary	0
hour_of_day	0
merchant_category	0
channel	0
customer_segment	0

Data types:

	data_type
date	datetime64[ns]
month_index	int64
transaction_amount	float64
transaction_frequency	int64
account_age_days	int64
customer_age	int64
device_risk_score	float64
location_risk_score	float64
merchant_risk_score	float64
failed_login_count	int64
beneficiary_age_days	int64
is_new_device	int64
is_new_beneficiary	int64
hour_of_day	int64
merchant_category	object
channel	object
customer_segment	object

=====
Sample Predictions
=====

Number of rows: 50000
Number of columns: 5
Missing values: 0
Duplicate rows: 3202

Missing values by column:

	missing_values
date	0
month_index	0
model_score	0
predicted_fraud	0
actual_fraud	0

Data types:

	data_type
date	datetime64[ns]
month_index	int64
model_score	float64
predicted_fraud	int64
actual_fraud	int64

=====
External Events
=====

Number of rows: 18
Number of columns: 4
Missing values: 0
Duplicate rows: 0

Missing values by column:

	missing_values
date	0
month_index	0
event_type	0
severity	0

Data types:

	data_type
--	-----------

```
# Aggregate transaction-level records by month

monthly_performance = (
    performance_log
    .groupby("date")
    .agg(
        total_transactions=("total_transactions", "sum"),
        actual_fraud_count=("actual_fraud", "sum"),
        predicted_fraud_count=("predicted_fraud", "sum"),
        true_positive_count=("true_positive", "sum"),
        false_positive_count=("false_positive", "sum"),
        true_negative_count=("true_negative", "sum"),
        false_negative_count=("false_negative", "sum")
    )
    .reset_index()
)

# Calculate monthly rates

monthly_performance["actual_fraud_rate"] = (
    monthly_performance["actual_fraud_count"]
    / monthly_performance["total_transactions"]
)

monthly_performance["predicted_fraud_rate"] = (
    monthly_performance["predicted_fraud_count"]
    / monthly_performance["total_transactions"]
)

monthly_performance["precision"] = (
    monthly_performance["true_positive_count"]
    / (
        monthly_performance["true_positive_count"]
        + monthly_performance["false_positive_count"]
    )
).fillna(0)

monthly_performance["recall"] = (
    monthly_performance["true_positive_count"]
    / (
        monthly_performance["true_positive_count"]
        + monthly_performance["false_negative_count"]
    )
).fillna(0)

monthly_performance["accuracy"] = (
    monthly_performance["true_positive_count"]
    + monthly_performance["true_negative_count"]
) / monthly_performance["total_transactions"]

monthly_performance
```

	date	total_transactions	actual_fraud_count	predicted_fraud_count	true_positive_count	false_positive_count	true_ne
0	2024-01-01	2693	312	0	0	0	
1	2024-02-01	2758	356	0	0	0	
2	2024-03-01	2774	336	0	0	0	
3	2024-04-01	2788	329	0	0	0	
4	2024-05-01	2791	339	0	0	0	
5	2024-06-01	2724	331	0	0	0	
6	2024-07-01	2781	377	1	0	1	
7	2024-08-01	2857	350	0	0	0	
8	2024-09-01	2771	309	0	0	0	
9	2024-10-01	2711	393	2	0	2	
10	2024-11-01	2743	404	0	0	0	
11	2024-12-01	2865	403	0	0	0	
12	2025-01-01	2829	421	0	0	0	
13	2025-02-01	2746	407	0	0	0	
14	2025-03-01	2794	430	1	0	1	
15	2025-04-01	2749	386	1	0	1	
16	2025-05-01	2794	404	0	0	0	
17	2025-06-01	2832	379	0	0	0	

Next steps:

[Generate code with monthly_performance](#)[New interactive sheet](#)

```
plt.figure(figsize=(12, 5))

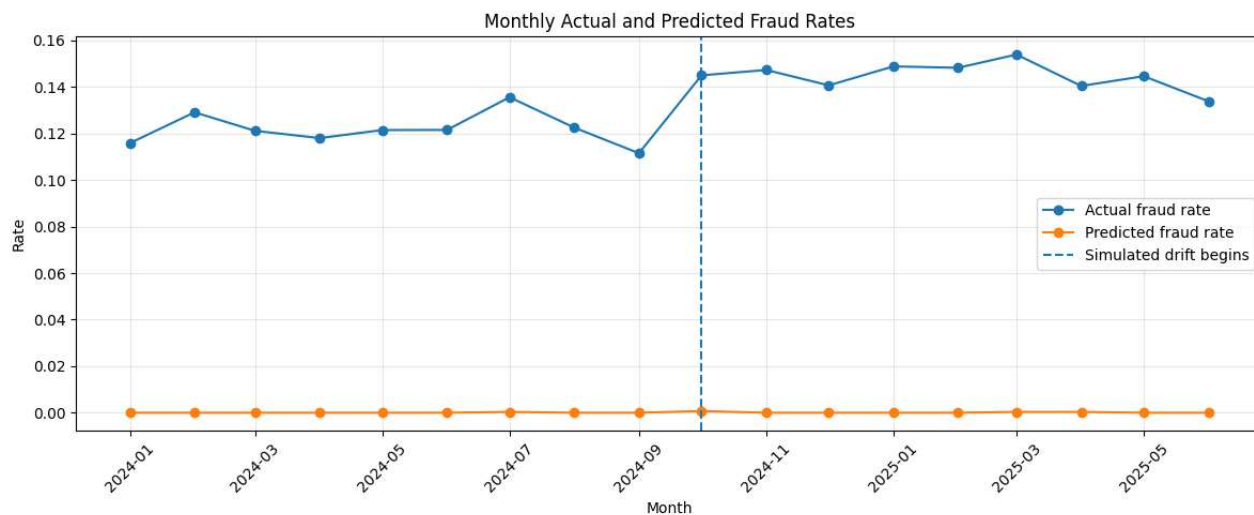
plt.plot(
    monthly_performance["date"],
    monthly_performance["actual_fraud_rate"],
    marker="o",
    label="Actual fraud rate"
)

plt.plot(
    monthly_performance["date"],
    monthly_performance["predicted_fraud_rate"],
    marker="o",
    label="Predicted fraud rate"
)

plt.axvline(
    pd.Timestamp("2024-10-01"),
    linestyle="--",
    label="Simulated drift begins"
)

plt.title("Monthly Actual and Predicted Fraud Rates")
plt.xlabel("Month")
plt.ylabel("Rate")
plt.legend()
plt.xticks(rotation=45)
plt.grid(True, alpha=0.3)
plt.tight_layout()
```

```
plt.show()
```



```
metrics_to_plot = [
    "accuracy",
    "precision",
    "recall"
]

plt.figure(figsize=(12, 6))

for metric in metrics_to_plot:
    plt.plot(
        monthly_performance["date"],
        monthly_performance[metric],
        marker="o",
        label=metric.capitalize()
    )

plt.axvline(
    pd.Timestamp("2024-10-01"),
    linestyle="--",
    label="Simulated drift begins"
)

plt.title("Monthly Model Performance")
plt.xlabel("Month")
plt.ylabel("Score")
plt.ylim(0, 1)
plt.legend()
plt.xticks(rotation=45)
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```



```
numeric_features = [
    "transaction_amount",
    "transaction_frequency",
    "account_age_days",
    "customer_age",
    "device_risk_score",
    "location_risk_score",
    "merchant_risk_score",
    "failed_login_count",
    "beneficiary_age_days",
    "hour_of_day"
]

feature_distributions[numeric_features].describe().T
```

	count	mean	std	min	25%	50%	75%	max
transaction_amount	50000.0	1509.464547	1455.467095	48.885768	634.090162	1085.743727	1875.197414	40352.016607
transaction_frequency	50000.0	4.995400	2.225443	0.000000	3.000000	5.000000	6.000000	19.000000
account_age_days	50000.0	1500.107780	864.472304	1.000000	754.000000	1497.000000	2250.000000	2999.000000
customer_age	50000.0	48.297460	17.860892	18.000000	33.000000	48.000000	64.000000	79.000000
device_risk_score	50000.0	0.285976	0.160390	0.001504	0.161027	0.263866	0.388428	0.948587
location_risk_score	50000.0	0.332616	0.177938	0.001600	0.193742	0.312709	0.452971	0.986913
merchant_risk_score	50000.0	0.285496	0.160142	0.001183	0.160141	0.264578	0.389975	0.892225
failed_login_count	50000.0	1.498860	1.225116	0.000000	1.000000	1.000000	2.000000	8.000000
beneficiary_age_days	50000.0	1004.080100	575.562464	1.000000	507.000000	1008.000000	1502.000000	1999.000000
hour_of_day	50000.0	11.527360	6.911466	0.000000	6.000000	12.000000	18.000000	23.000000

```
numeric_features = [
    "transaction_amount",
    "transaction_frequency",
    "account_age_days",
    "customer_age",
    "device_risk_score",
    "location_risk_score",
    "merchant_risk_score",
    "failed_login_count",
    "beneficiary_age_days",
    "hour_of_day"
]

feature_distributions[numeric_features].describe().T
```

	count	mean	std	min	25%	50%	75%	max	
transaction_amount	50000.0	1509.464547	1455.467095	48.885768	634.090162	1085.743727	1875.197414	40352.016607	
transaction_frequency	50000.0	4.995400	2.225443	0.000000	3.000000	5.000000	6.000000	19.000000	
account_age_days	50000.0	1500.107780	864.472304	1.000000	754.000000	1497.000000	2250.000000	2999.000000	
customer_age	50000.0	48.297460	17.860892	18.000000	33.000000	48.000000	64.000000	79.000000	
device_risk_score	50000.0	0.285976	0.160390	0.001504	0.161027	0.263866	0.388428	0.948587	
location_risk_score	50000.0	0.332616	0.177938	0.001600	0.193742	0.312709	0.452971	0.986913	
merchant_risk_score	50000.0	0.285496	0.160142	0.001183	0.160141	0.264578	0.389975	0.892225	
failed_login_count	50000.0	1.498860	1.225116	0.000000	1.000000	1.000000	2.000000	8.000000	
beneficiary_age_days	50000.0	1004.080100	575.562464	1.000000	507.000000	1008.000000	1502.000000	1999.000000	
hour_of_day	50000.0	11.527360	6.911466	0.000000	6.000000	12.000000	18.000000	23.000000	

```

selected_features = [
    "transaction_amount",
    "device_risk_score",
    "location_risk_score",
    "merchant_risk_score"
]

for feature in selected_features:
    plt.figure(figsize=(8, 4))

    sns.histplot(
        data=feature_distributions,
        x=feature,
        bins=40,
        kde=True
    )

    plt.title(f"Distribution of {feature}")
    plt.xlabel(feature)
    plt.ylabel("Count")
    plt.tight_layout()
    plt.show()

```



```

baseline_end = pd.Timestamp("2024-06-30")

baseline_data = feature_distributions[
    feature_distributions["date"] <= baseline_end
].copy()

later_data = feature_distributions[
    feature_distributions["date"] > baseline_end
].copy()

print("Baseline records:", len(baseline_data))
print("Later-period records:", len(later_data))

```

Baseline records: 15528
Later-period records: 33472

```

comparison_rows = []

for feature in numeric_features:
    baseline_mean = baseline_data[feature].mean()
    later_mean = later_data[feature].mean()

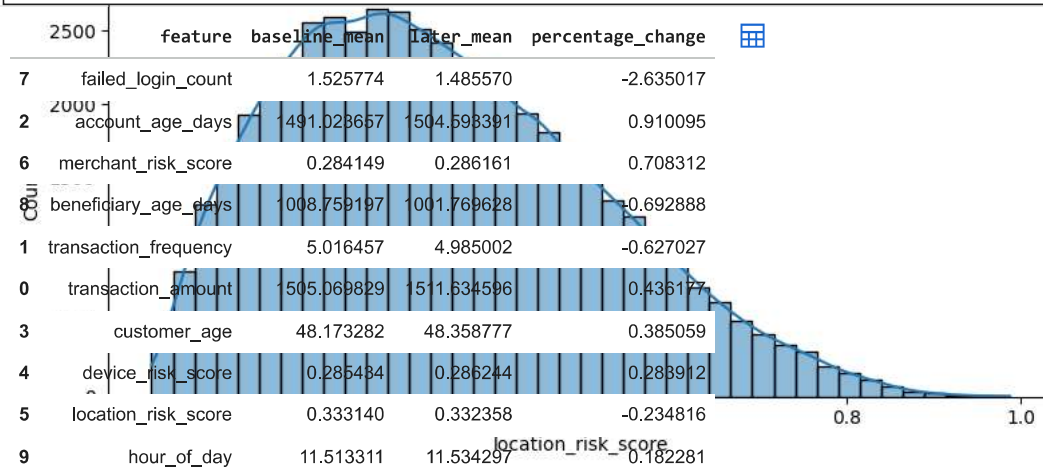
    percentage_change = (
        (later_mean - baseline_mean)
        / abs(baseline_mean)
        * 100
    )

    comparison_rows.append({
        "feature": feature,
        "baseline_mean": baseline_mean,
        "later_mean": later_mean,
        "percentage_change": percentage_change
    })

feature_comparison = pd.DataFrame(comparison_rows)

feature_comparison.sort_values(
    by="percentage_change",
    key=lambda column: column.abs(),
    ascending=False
)

```



Distribution of merchant_risk_score

```

baseline_results_path = (
    REPORT_DIR / "baseline_monthly_performance.csv"
)

monthly_performance.to_csv(
    baseline_results_path,
    index=False
)

feature_comparison.to_csv(
    REPORT_DIR / "baseline_feature_comparison.csv",
    index=False
)

print("Baseline results saved successfully.")

```

Baseline results saved successfully.

merchant risk score

```
# Step 3.1: Define baseline and comparison periods

baseline_start = pd.Timestamp("2024-01-01")
baseline_end = pd.Timestamp("2024-06-30")

comparison_start = pd.Timestamp("2024-07-01")

baseline_data = feature_distributions[
    (feature_distributions["date"] >= baseline_start)
    & (feature_distributions["date"] <= baseline_end)
].copy()

comparison_data = feature_distributions[
    feature_distributions["date"] >= comparison_start
].copy()

print("Baseline period:")
print(baseline_data["date"].min(), "to", baseline_data["date"].max())

print("\nComparison period:")
print(comparison_data["date"].min(), "to", comparison_data["date"].max())

print("\nBaseline rows:", len(baseline_data))
print("Comparison rows:", len(comparison_data))
```

Baseline period:
2024-01-01 00:00:00 to 2024-06-01 00:00:00

Comparison period:
2024-07-01 00:00:00 to 2025-06-01 00:00:00

Baseline rows: 16528
Comparison rows: 33472

```
# Step 3.2: Kolmogorov-Smirnov test

from scipy.stats import ks_2samp

ks_results = []

for feature in numeric_features:
    baseline_values = baseline_data[feature].dropna()
    comparison_values = comparison_data[feature].dropna()

    ks_statistic, p_value = ks_2samp(
        baseline_values,
        comparison_values
    )

    ks_results.append({
        "feature": feature,
        "ks_statistic": ks_statistic,
        "p_value": p_value,
        "drift_detected": p_value < 0.05
    })

ks_results_df = pd.DataFrame(ks_results)

ks_results_df = ks_results_df.sort_values(
    by="ks_statistic",
    ascending=False
)

display(ks_results_df)
```

	feature	ks_statistic	p_value	drift_detected
7	failed_login_count	0.014094	0.024424	True
2	account_age_days	0.011427	0.110302	False
4	device_risk_score	0.011076	0.131433	False
8	beneficiary_age_days	0.008827	0.352501	False
0	transaction_amount	0.007211	0.610192	False
6	merchant_risk_score	0.007100	0.629754	False
5	location_risk_score	0.005831	0.843938	False
3	customer_age	0.005054	0.938713	False
1	transaction_frequency	0.004919	0.950476	False
9	hour_of_day	0.003581	0.998806	False

Next steps: [Generate code with ks_results_df](#) [New interactive sheet](#)

Step 3.3: Population Stability Index

```
def calculate_psi(expected, actual, bins=10):
    expected = pd.Series(expected).dropna()
    actual = pd.Series(actual).dropna()

    # Create bins from the baseline distribution
    breakpoints = np.unique(
        np.quantile(
            expected,
            np.linspace(0, 1, bins + 1)
        )
    )

    # Handle features with few unique values
    if len(breakpoints) < 3:
        breakpoints = np.linspace(
            expected.min(),
            expected.max(),
            bins + 1
        )

    # Extend the edges so comparison values are not lost
    breakpoints[0] = -np.inf
    breakpoints[-1] = np.inf

    expected_counts, _ = np.histogram(
        expected,
        bins=breakpoints
    )

    actual_counts, _ = np.histogram(
        actual,
        bins=breakpoints
    )

    # Convert counts into proportions
    expected_percentages = (
        expected_counts + 0.0001
    ) / (
        expected_counts.sum()
        + 0.0001 * len(expected_counts)
    )

    actual_percentages = (
        actual_counts + 0.0001
    ) / (
        actual_counts.sum()
        + 0.0001 * len(actual_counts)
    )

    psi_value = np.sum(
        (
            actual_percentages
            - expected_percentages
        )
        * np.log(
            actual_percentages
            / expected_percentages
        )
    )
```

```
)
)

return float(psi_value)

print("PSI function created successfully.")
```

PSI function created successfully.

```
test_psi = calculate_psi(
    [10, 20, 30, 40, 50],
    [10, 20, 30, 40, 50]
)

print("PSI for identical distributions:", test_psi)
```

PSI for identical distributions: 0.0

```
# Calculate PSI for all numerical features

psi_results = []

for feature in numeric_features:
    psi_value = calculate_psi(
        baseline_data[feature],
        comparison_data[feature]
    )

    if psi_value < 0.10:
        interpretation = "Little or no drift"
    elif psi_value < 0.25:
        interpretation = "Moderate drift"
    else:
        interpretation = "Significant drift"

    psi_results.append({
        "feature": feature,
        "psi": psi_value,
        "interpretation": interpretation,
        "drift_detected": psi_value >= 0.10
    })

psi_results_df = pd.DataFrame(psi_results)

psi_results_df = psi_results_df.sort_values(
    by="psi",
    ascending=False
)

display(psi_results_df)
```

	feature	psi	interpretation	drift_detected	
7	failed_login_count	0.001275	Little or no drift	False	 
2	account_age_days	0.001056	Little or no drift	False	
8	beneficiary_age_days	0.000728	Little or no drift	False	
6	merchant_risk_score	0.000704	Little or no drift	False	
0	transaction_amount	0.000603	Little or no drift	False	
3	customer_age	0.000603	Little or no drift	False	
5	location_risk_score	0.000557	Little or no drift	False	
4	device_risk_score	0.000531	Little or no drift	False	
1	transaction_frequency	0.000443	Little or no drift	False	
9	hour_of_day	0.000435	Little or no drift	False	

Next steps: [Generate code with psi_results_df](#) [New interactive sheet](#)

```
# Step 3.4: Chi-square test for categorical features

from scipy.stats import chi2_contingency

categorical_features = [
    "merchant_category",
    "channel",
```

```

    "customer_segment"
]

chi_square_results = []

for feature in categorical_features:

    baseline_counts = (
        baseline_data[feature]
        .value_counts()
    )

    comparison_counts = (
        comparison_data[feature]
        .value_counts()
    )

    contingency_table = pd.DataFrame({
        "baseline": baseline_counts,
        "comparison": comparison_counts
    }).fillna(0)

    chi2_statistic, p_value, degrees_freedom, expected = (
        chi2_contingency(contingency_table)
    )

    chi_square_results.append({
        "feature": feature,
        "chi2_statistic": chi2_statistic,
        "degrees_freedom": degrees_freedom,
        "p_value": p_value,
        "drift_detected": p_value < 0.05
    })

chi_square_results_df = pd.DataFrame(
    chi_square_results
)

display(chi_square_results_df)

```

	feature	chi2_statistic	degrees_freedom	p_value	drift_detected	
0	merchant_category	0.317216	5	0.997307	False	
1	channel	1.503063	3	0.681564	False	
2	customer_segment	4.903689	2	0.086135	False	

Next steps: [Generate code with chi_square_results_df](#) [New interactive sheet](#)

```

# Step 3.5: PSI visualization

import matplotlib.pyplot as plt

plt.figure(figsize=(12, 6))

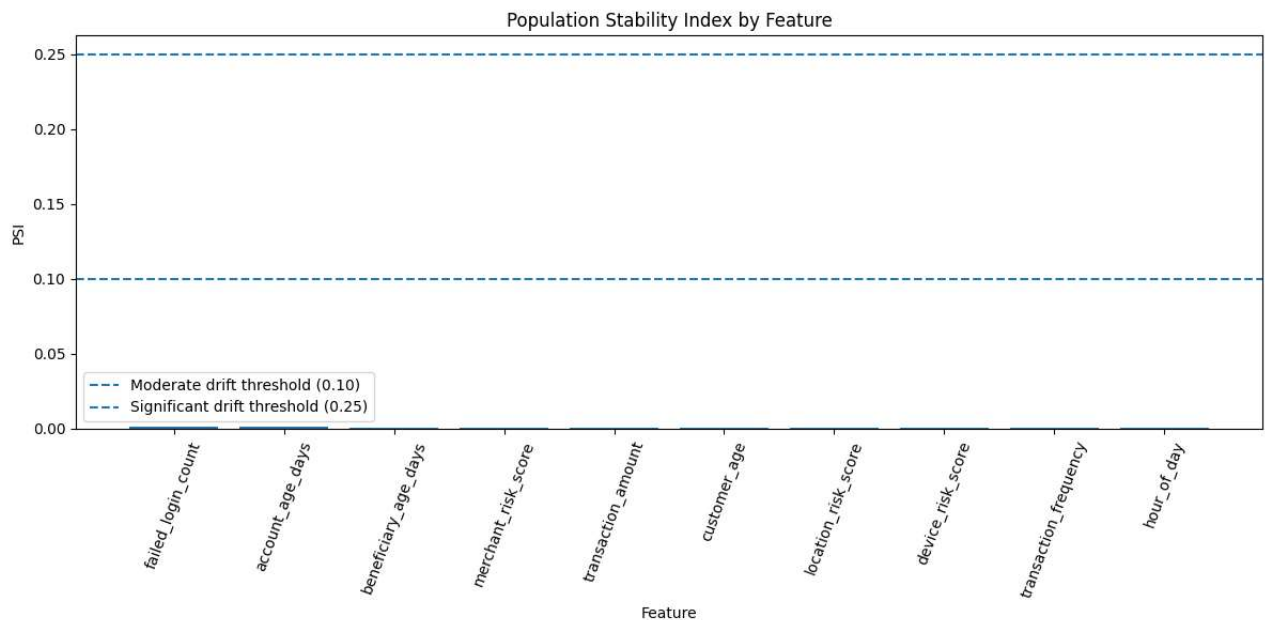
plt.bar(
    psi_results_df["feature"],
    psi_results_df["psi"]
)

plt.axhline(
    0.10,
    linestyle="--",
    label="Moderate drift threshold (0.10)"
)

plt.axhline(
    0.25,
    linestyle="--",
    label="Significant drift threshold (0.25)"
)

plt.title("Population Stability Index by Feature")
plt.xlabel("Feature")
plt.ylabel("PSI")
plt.xticks(rotation=70)
plt.legend()
plt.tight_layout()
plt.show()

```



Step 3.6: Combined numerical drift report

```
numerical_drift_report = pd.merge(
    ks_results_df,
    psi_results_df,
    on="feature",
    how="outer",
    suffixes=("_ks", "_psi")
)

numerical_drift_report["overall_drift_flag"] = (
    numerical_drift_report["drift_detected_ks"]
    | numerical_drift_report["drift_detected_psi"]
)

display(
    numerical_drift_report.sort_values(
        by="psi",
        ascending=False
    )
)
```

	feature	ks_statistic	p_value	drift_detected_ks	psi	interpretation	drift_detected_psi	overall_drift
4	failed_login_count	0.014094	0.024424	True	0.001275	Little or no drift	False	
0	account_age_days	0.011427	0.110302	False	0.001056	Little or no drift	False	
1	beneficiary_age_days	0.008827	0.352501	False	0.000728	Little or no drift	False	
7	merchant_risk_score	0.007100	0.629754	False	0.000704	Little or no drift	False	
8	transaction_amount	0.007211	0.610192	False	0.000603	Little or no drift	False	
2	customer_age	0.005054	0.938713	False	0.000603	Little or no drift	False	
6	location_risk_score	0.005831	0.843938	False	0.000557	Little or no drift	False	
3	device_risk_score	0.011076	0.131433	False	0.000531	Little or no drift	False	
9	transaction_frequency	0.004919	0.950476	False	0.000443	Little or no drift	False	
5	hour_of_day	0.003581	0.998806	False	0.000435	Little or no drift	False	

Step 3.7: Save drift reports

```
numerical_drift_report.to_csv(
    REPORT_DIR / "numerical_drift_report.csv",
    index=False
)

chi_square_results_df.to_csv(
```

```
REPORT_DIR / "categorical_drift_report.csv",
index=False
)

print("Step 3 reports saved successfully.")
```

Step 3 reports saved successfully.

```
# Step 4.1: Jensen-Shannon Divergence

from scipy.spatial.distance import jensenshannon
import numpy as np
import pandas as pd

def calculate_js_divergence(expected, actual, bins=10):

    expected = pd.Series(expected).dropna()
    actual = pd.Series(actual).dropna()

    # Create common bins using both distributions
    combined = pd.concat([expected, actual])

    breakpoints = np.unique(
        np.quantile(
            combined,
            np.linspace(0, 1, bins + 1)
        )
    )

    # Handle constant or nearly constant features
    if len(breakpoints) < 3:
        return 0.0

    breakpoints[0] = -np.inf
    breakpoints[-1] = np.inf

    expected_counts, _ = np.histogram(
        expected,
        bins=breakpoints
    )

    actual_counts, _ = np.histogram(
        actual,
        bins=breakpoints
    )

    # Convert counts to probability distributions
    expected_prob = (
        expected_counts + 0.0001
    ) / (
        expected_counts.sum()
        + 0.0001 * len(expected_counts)
    )

    actual_prob = (
        actual_counts + 0.0001
    ) / (
        actual_counts.sum()
        + 0.0001 * len(actual_counts)
    )

    # Calculate JS divergence
    js_distance = jensenshannon(
        expected_prob,
        actual_prob,
        base=2
    )

    js_divergence = js_distance ** 2

    return float(js_divergence)

print("JS divergence function created successfully.")
```

JS divergence function created successfully.

```
# Step 4.2: Test JS divergence

test_js = calculate_js_divergence(
    [10, 20, 30, 40, 50],
```

```
[10, 20, 30, 40, 50]
)

print("JS divergence for identical distributions:", test_js)
```

JS divergence for identical distributions: 0.0

Step 4.3: Calculate JS divergence for all numerical features

```
js_results = []

for feature in numeric_features:

    js_value = calculate_js_divergence(
        baseline_data[feature],
        comparison_data[feature]
    )

    js_results.append({
        "feature": feature,
        "js_divergence": js_value
    })

js_results_df = pd.DataFrame(js_results)

js_results_df = js_results_df.sort_values(
    by="js_divergence",
    ascending=False
)

display(js_results_df)
```

	feature	js_divergence	
7	failed_login_count	0.000230	
8	beneficiary_age_days	0.000161	
2	account_age_days	0.000144	
6	merchant_risk_score	0.000135	
0	transaction_amount	0.000119	
4	device_risk_score	0.000111	
3	customer_age	0.000106	
1	transaction_frequency	0.000080	
9	hour_of_day	0.000078	
5	location_risk_score	0.000075	

Next steps: [Generate code with js_results_df](#) [New interactive sheet](#)

```
# Save JS divergence results

js_results_df.to_csv(
    REPORT_DIR / "js_divergence_report.csv",
    index=False
)

print("JS divergence report saved successfully.")
```

JS divergence report saved successfully.

Step 4.4: Combined advanced drift report

```
advanced_drift_report = ks_results_df[
    ["feature", "ks_statistic", "p_value", "drift_detected"]
].merge(
    psi_results_df[
        ["feature", "psi", "interpretation"]
    ],
    on="feature",
    how="left"
).merge(
    js_results_df,
    on="feature",
    how="left"
)
```

```
# Rename columns for clarity
advanced_drift_report = advanced_drift_report.rename(
    columns={
        "drift_detected": "ks_drift_detected"
    }
)

display(
    advanced_drift_report.sort_values(
        by="js_divergence",
        ascending=False
    )
)
```

	feature	ks_statistic	p_value	ks_drift_detected	psi	interpretation	js_divergence	
0	failed_login_count	0.014094	0.024424	True	0.001275	Little or no drift	0.000230	
3	beneficiary_age_days	0.008827	0.352501	False	0.000728	Little or no drift	0.000161	
1	account_age_days	0.011427	0.110302	False	0.001056	Little or no drift	0.000144	
5	merchant_risk_score	0.007100	0.629754	False	0.000704	Little or no drift	0.000135	
4	transaction_amount	0.007211	0.610192	False	0.000603	Little or no drift	0.000119	
2	device_risk_score	0.011076	0.131433	False	0.000531	Little or no drift	0.000111	
7	customer_age	0.005054	0.938713	False	0.000603	Little or no drift	0.000106	
8	transaction_frequency	0.004919	0.950476	False	0.000443	Little or no drift	0.000080	
9	hour_of_day	0.003581	0.998806	False	0.000435	Little or no drift	0.000078	
6	location_risk_score	0.005831	0.843938	False	0.000557	Little or no drift	0.000075	

```
# Step 4.5: Drift severity classification

def classify_drift(row):
    if row["ks_drift_detected"] or row["psi"] >= 0.25:
        return "High"
    elif row["psi"] >= 0.10:
        return "Moderate"
    else:
        return "Low"

advanced_drift_report["drift_severity"] = (
    advanced_drift_report.apply(
        classify_drift,
        axis=1
    )
)

display(
    advanced_drift_report[
        [
            "feature",
            "ks_statistic",
            "p_value",
            "psi",
            "js_divergence",
            "drift_severity"
        ]
    ]
)
```

	feature	ks_statistic	p_value	psi	js_divergence	drift_severity
0	failed_login_count	0.014094	0.024424	0.001275	0.000230	High
1	account_age_days	0.011427	0.110302	0.001056	0.000144	Low
2	device_risk_score	0.011076	0.131433	0.000531	0.000111	Low
3	beneficiary_age_days	0.008827	0.352501	0.000728	0.000161	Low
4	transaction_amount	0.007211	0.610192	0.000603	0.000119	Low
5	merchant_risk_score	0.007100	0.629754	0.000704	0.000135	Low
6	location_risk_score	0.005831	0.843938	0.000557	0.000075	Low
7	customer_age	0.005054	0.938713	0.000603	0.000106	Low
8	transaction_frequency	0.004919	0.950476	0.000443	0.000080	Low
9	hour_of_day	0.003581	0.998806	0.000435	0.000078	Low

```
# Save advanced drift report
```

```
advanced_drift_report.to_csv(
    REPORT_DIR / "advanced_drift_report.csv",
    index=False
)

print("Advanced drift report saved successfully.")
```

Advanced drift report saved successfully.

```
# Step 4.6: Drift severity visualization
```

```
import matplotlib.pyplot as plt

severity_counts = (
    advanced_drift_report["drift_severity"]
    .value_counts()
    .reindex(["Low", "Moderate", "High"], fill_value=0)
)

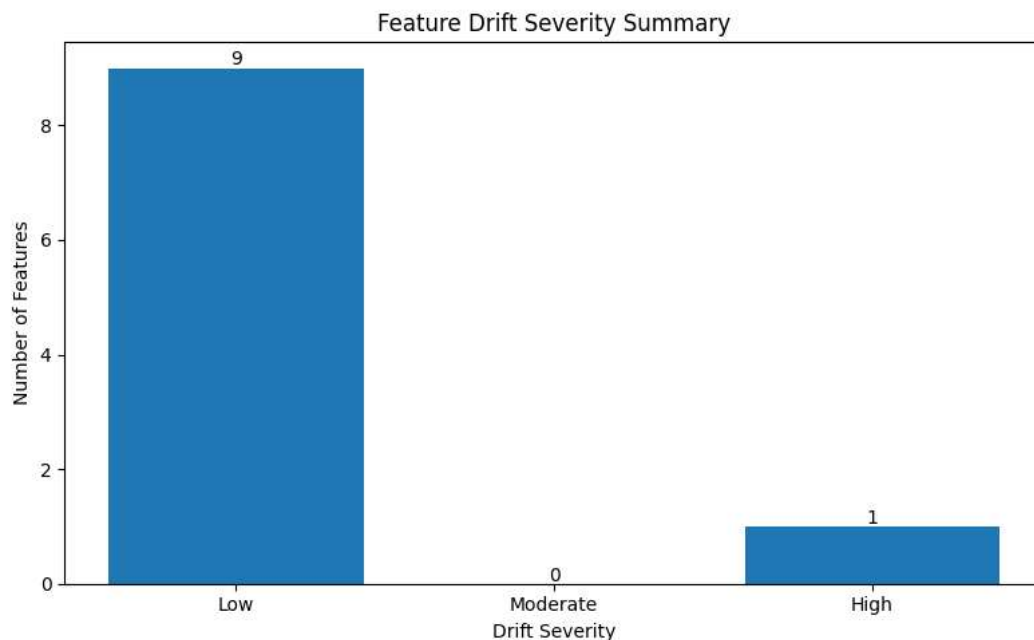
plt.figure(figsize=(8, 5))

plt.bar(
    severity_counts.index,
    severity_counts.values
)

plt.title("Feature Drift Severity Summary")
plt.xlabel("Drift Severity")
plt.ylabel("Number of Features")

for i, value in enumerate(severity_counts.values):
    plt.text(i, value + 0.05, str(value), ha="center")

plt.tight_layout()
plt.show()
```



```
# Step 5.1: Inspect sample predictions dataset
```

```
print("Sample predictions columns:")  
print(sample_predictions.columns.tolist())
```

```
print("\nDataset shape:")  
print(sample_predictions.shape)
```

```
print("\nFirst 5 rows:")  
display(sample_predictions.head())
```

```
print("\nData types:")  
display(sample_predictions.dtypes)
```

Sample predictions columns:

['date', 'month_index', 'model_score', 'predicted_fraud', 'actual_fraud']

Dataset shape:
(50000, 5)

First 5 rows:

	date	month_index	model_score	predicted_fraud	actual_fraud
0	2025-02-01	13	0.038600	0	0
1	2025-04-01	15	0.109725	0	0
2	2024-11-01	10	0.129085	0	0
3	2024-08-01	7	0.172345	0	0
4	2024-01-01	0	0.047204	0	0

Data types:

	0
date	datetime64[ns]
month_index	int64
model_score	float64
predicted_fraud	int64
actual_fraud	int64

dtype: object

```
# Step 5.2: Monthly model performance analysis
```

```
from sklearn.metrics import (  
    precision_score,  
    recall_score,  
    f1_score  
)
```

```
# Make a copy of the prediction dataset
```

```
predictions = sample_predictions.copy()

# Ensure date is in datetime format
predictions["date"] = pd.to_datetime(
    predictions["date"]
)

# Sort data by date
predictions = predictions.sort_values("date")

# Calculate monthly performance
monthly_performance = []

for month, group in predictions.groupby("date"):

    actual = group["actual_fraud"]
    predicted = group["predicted_fraud"]

    precision = precision_score(
        actual,
        predicted,
        zero_division=0
    )

    recall = recall_score(
        actual,
        predicted,
        zero_division=0
    )

    f1 = f1_score(
        actual,
        predicted,
        zero_division=0
    )

    monthly_performance.append({
        "date": month,
        "precision": precision,
        "recall": recall,
        "f1_score": f1,
        "actual_fraud_rate": actual.mean(),
        "prediction_rate": predicted.mean(),
        "total_transactions": len(group)
    })

monthly_performance_df = pd.DataFrame(
    monthly_performance
)

display(monthly_performance_df)
```

	date	precision	recall	f1_score	actual_fraud_rate	prediction_rate	total_transactions
0	2024-01-01	0.0	0.0	0.0	0.115856	0.000000	2693
1	2024-02-01	0.0	0.0	0.0	0.129079	0.000000	2758
2	2024-03-01	0.0	0.0	0.0	0.121125	0.000000	2774
3	2024-04-01	0.0	0.0	0.0	0.118006	0.000000	2788
4	2024-05-01	0.0	0.0	0.0	0.121462	0.000000	2791
5	2024-06-01	0.0	0.0	0.0	0.121512	0.000000	2724
6	2024-07-01	0.0	0.0	0.0	0.135563	0.000360	2781
7	2024-08-01	0.0	0.0	0.0	0.122506	0.000000	2857
8	2024-09-01	0.0	0.0	0.0	0.111512	0.000000	2771
9	2024-10-01	0.0	0.0	0.0	0.144965	0.000738	2711
10	2024-11-01	0.0	0.0	0.0	0.147284	0.000000	2743
11	2024-12-01	0.0	0.0	0.0	0.140663	0.000000	2865
12	2025-01-01	0.0	0.0	0.0	0.148816	0.000000	2829
13	2025-02-01	0.0	0.0	0.0	0.148216	0.000000	2746
14	2025-03-01	0.0	0.0	0.0	0.153901	0.000358	2794
15	2025-04-01	0.0	0.0	0.0	0.140415	0.000364	2749
16	2025-05-01	0.0	0.0	0.0	0.144596	0.000000	2794
17	2025-06-01	0.0	0.0	0.0	0.133828	0.000000	2832

Next steps:

[Generate code with monthly_performance_df](#)[New interactive sheet](#)

Step 5.3: Model performance trend visualization

```
import matplotlib.pyplot as plt

plt.figure(figsize=(12, 6))

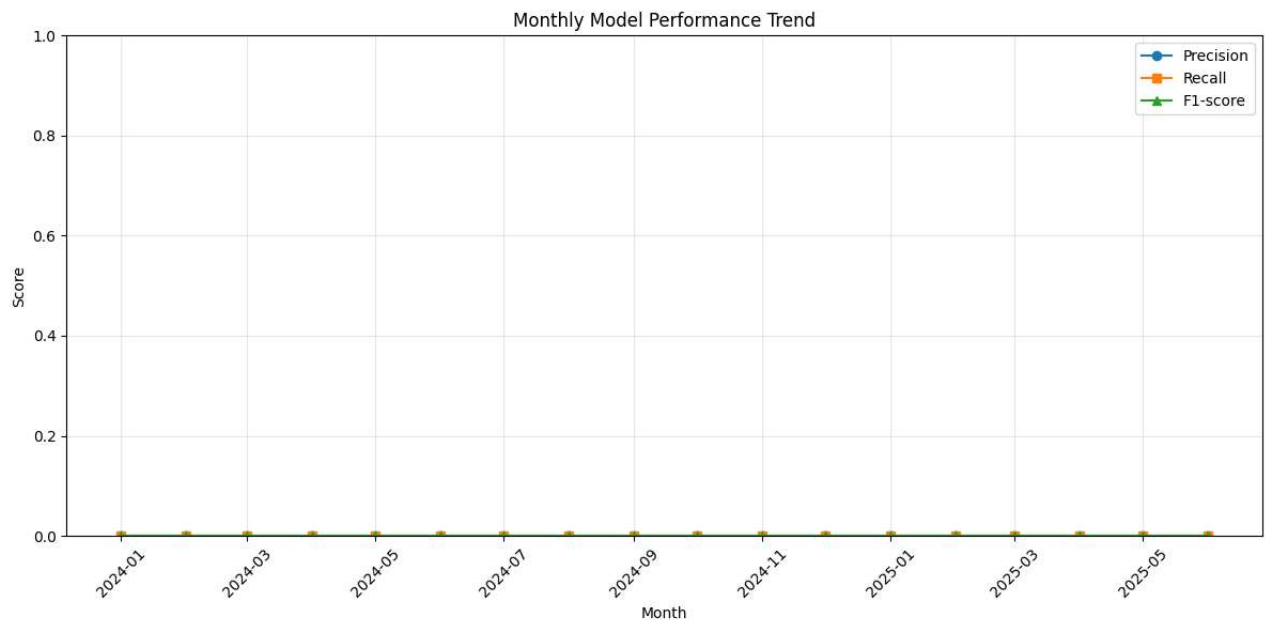
plt.plot(
    monthly_performance_df["date"],
    monthly_performance_df["precision"],
    marker="o",
    label="Precision"
)

plt.plot(
    monthly_performance_df["date"],
    monthly_performance_df["recall"],
    marker="s",
    label="Recall"
)

plt.plot(
    monthly_performance_df["date"],
    monthly_performance_df["f1_score"],
    marker="^",
    label="F1-score"
)

plt.title("Monthly Model Performance Trend")
plt.xlabel("Month")
plt.ylabel("Score")
plt.ylim(0, 1)
plt.xticks(rotation=45)
plt.legend()
plt.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()
```



```
# Step 5.4: Fraud prediction threshold analysis

import numpy as np
import pandas as pd
from sklearn.metrics import (
    precision_score,
    recall_score,
    f1_score,
    accuracy_score
)

# Use the existing predictions dataset
threshold_data = sample_predictions.copy()

# Check actual fraud distribution
print("Actual fraud distribution:")
print(threshold_data["actual_fraud"].value_counts())

print("\nModel score summary:")
display(threshold_data["model_score"].describe())

# Test different probability thresholds
thresholds = np.arange(0.01, 1.00, 0.01)

threshold_results = []

for threshold in thresholds:

    predicted = (
        threshold_data["model_score"] >= threshold
    ).astype(int)

    actual = threshold_data["actual_fraud"]

    threshold_results.append({
        "threshold": round(threshold, 2),
        "predicted_fraud_count": predicted.sum(),
        "actual_fraud_count": actual.sum(),
        "precision": precision_score(
            actual, predicted, zero_division=0
        ),
        "recall": recall_score(
            actual, predicted, zero_division=0
        ),
        "f1_score": f1_score(
            actual, predicted, zero_division=0
        ),
        "accuracy": accuracy_score(
            actual, predicted
        )
    })
```

```

    })

threshold_results_df = pd.DataFrame(
    threshold_results
)

print("\nThreshold analysis results:")
display(threshold_results_df.head(10))

print("\nBest threshold by F1-score:")
display(
    threshold_results_df.sort_values(
        "f1_score",
        ascending=False
    ).head(10)
)

```

Actual fraud distribution:

```

actual_fraud
0    43334
1     6666
Name: count, dtype: int64

```

Model score summary:

	model_score
count	50000.000000
mean	0.133796
std	0.084036
min	0.000000
25%	0.070948
50%	0.129404
75%	0.190287
max	0.555120

dtype: float64

Threshold analysis results:

	threshold	predicted_fraud_count	actual_fraud_count	precision	recall	f1_score	accuracy
0	0.01	45983	6666	0.136311	0.940294	0.238105	0.19774
1	0.02	45018	6666	0.137212	0.926643	0.239029	0.21340
2	0.03	43800	6666	0.137831	0.905641	0.239250	0.23216
3	0.04	42421	6666	0.138634	0.882238	0.239615	0.25350
4	0.05	40975	6666	0.139109	0.855086	0.239290	0.27518
5	0.06	39415	6666	0.140175	0.828833	0.239795	0.29938
6	0.07	37700	6666	0.140849	0.796580	0.239372	0.32508
7	0.08	35744	6666	0.141758	0.760126	0.238953	0.35448
8	0.09	33769	6666	0.142527	0.722022	0.238061	0.38382
9	0.10	31640	6666	0.143394	0.680618	0.236882	0.41536

Best threshold by F1-score:

	threshold	predicted_fraud_count	actual_fraud_count	precision	recall	f1_score	accuracy
5	0.06	39415	6666	0.140175	0.828833	0.239795	0.29938
3	0.04	42421	6666	0.138634	0.882238	0.239615	0.25350
6	0.07	37700	6666	0.140849	0.796580	0.239372	0.32508
4	0.05	40975	6666	0.139109	0.855086	0.239290	0.27518
2	0.03	43800	6666	0.137831	0.905641	0.239250	0.23216
1	0.02	45018	6666	0.137212	0.926643	0.239029	0.21340
7	0.08	35744	6666	0.141758	0.760126	0.238953	0.35448
0	0.01	45983	6666	0.136311	0.940294	0.238105	0.19774
8	0.09	33769	6666	0.142527	0.722022	0.238061	0.38382
9	0.10	31640	6666	0.143394	0.680618	0.236882	0.41536

```
# Step 5.5: Baseline vs comparison performance

from sklearn.metrics import (
    precision_score,
    recall_score,
    f1_score,
    accuracy_score
)

# Use the threshold identified in Step 5.4
best_threshold = 0.06

# Prepare the dataset
performance_data = sample_predictions.copy()
performance_data["date"] = pd.to_datetime(
    performance_data["date"]
)

# Generate predictions using the selected threshold
performance_data["threshold_prediction"] = (
    performance_data["model_score"] >= best_threshold
).astype(int)

# Define baseline and comparison periods
baseline_period = performance_data[
    (performance_data["date"] >= "2024-01-01") &
    (performance_data["date"] <= "2024-06-01")
]

comparison_period = performance_data[
    (performance_data["date"] >= "2024-07-01") &
    (performance_data["date"] <= "2025-06-01")
]

# Calculate performance metrics
def calculate_metrics(data, period_name):

    actual = data["actual_fraud"]
    predicted = data["threshold_prediction"]

    return {
        "period": period_name,
        "total_transactions": len(data),
        "actual_fraud_count": actual.sum(),
        "predicted_fraud_count": predicted.sum(),
        "precision": precision_score(
            actual, predicted, zero_division=0
        ),
        "recall": recall_score(
            actual, predicted, zero_division=0
        ),
        "f1_score": f1_score(
            actual, predicted, zero_division=0
        ),
        "accuracy": accuracy_score(
            actual, predicted
        )
    }

performance_comparison = pd.DataFrame([
    calculate_metrics(baseline_period, "Baseline"),
    calculate_metrics(comparison_period, "Comparison")
])

display(performance_comparison)
```

	period	total_transactions	actual_fraud_count	predicted_fraud_count	precision	recall	f1_score	accuracy	
0	Baseline	16528	2003	12511	0.127488	0.796306	0.219788	0.314860	
1	Comparison	33472	4663	26904	0.146075	0.842805	0.248994	0.291736	

Next steps: [Generate code with performance_comparison](#) [New interactive sheet](#)

```
# Step 6.1: Record baseline model performance

baseline_metrics = pd.DataFrame([
    calculate_metrics(baseline_period, "Baseline")
])
```

```
print("Baseline Model Performance:")
display(baseline_metrics)

# Save baseline metrics for later comparison
baseline_metrics.to_csv(
    "baseline_performance_metrics.csv",
    index=False
)

print("Baseline metrics saved successfully.")
```

Baseline Model Performance:

	period	total_transactions	actual_fraud_count	predicted_fraud_count	precision	recall	f1_score	accuracy	
0	Baseline	16528	2003	12511	0.127488	0.796306	0.219788	0.31486	

Baseline metrics saved successfully.

Step 6.1: Investigate possible causes of drift

```
import pandas as pd
import matplotlib.pyplot as plt

# Compare fraud rates between baseline and comparison periods
cause_analysis = pd.DataFrame({
    "Metric": [
        "Average model score",
        "Actual fraud rate",
        "Predicted fraud rate"
    ],
    "Baseline": [
        baseline_period["model_score"].mean(),
        baseline_period["actual_fraud"].mean(),
        baseline_period["predicted_fraud"].mean()
    ],
    "Comparison": [
        comparison_period["model_score"].mean(),
        comparison_period["actual_fraud"].mean(),
        comparison_period["predicted_fraud"].mean()
    ]
})

cause_analysis["Change"] = (
    cause_analysis["Comparison"] - cause_analysis["Baseline"]
)

display(cause_analysis)
```

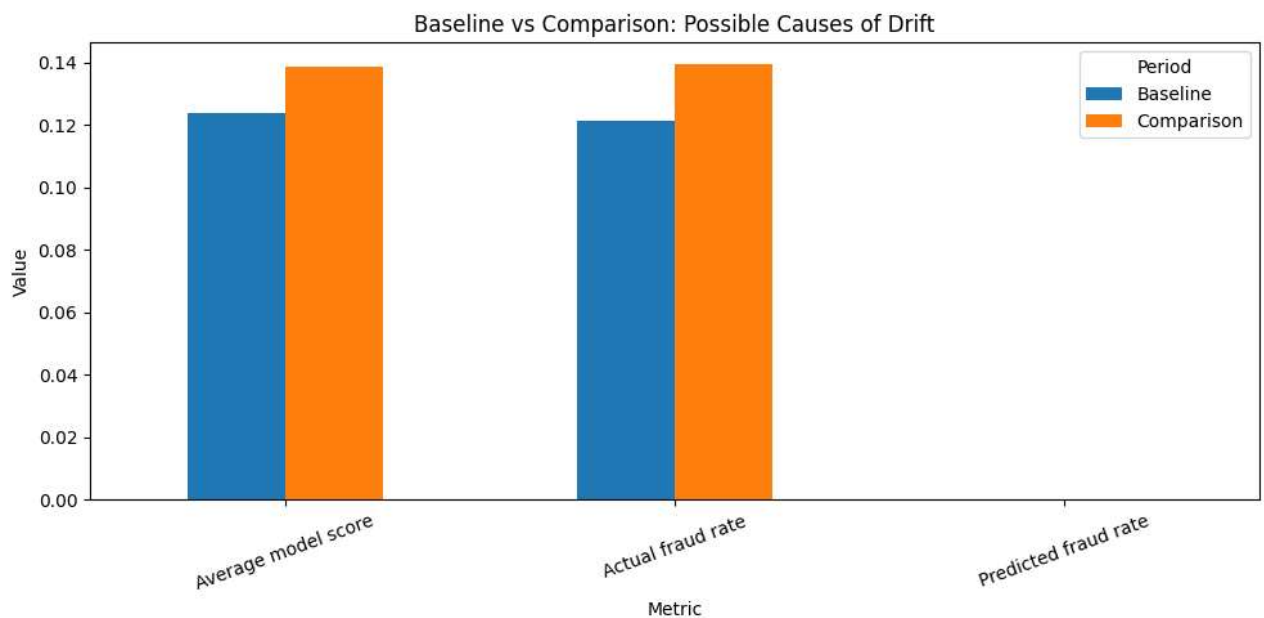
	Metric	Baseline	Comparison	Change	
0	Average model score	0.123876	0.138695	0.014819	
1	Actual fraud rate	0.121188	0.139310	0.018122	
2	Predicted fraud rate	0.000000	0.000149	0.000149	

Next steps: [Generate code with cause_analysis](#) [New interactive sheet](#)

```
# Visualise baseline vs comparison metrics

cause_analysis.set_index("Metric")[["Baseline", "Comparison"]].plot(
    kind="bar",
    figsize=(10, 5)
)

plt.title("Baseline vs Comparison: Possible Causes of Drift")
plt.ylabel("Value")
plt.xticks(rotation=20)
plt.legend(title="Period")
plt.tight_layout()
plt.show()
```



```
# Step 6.2: Compare feature averages

features_to_check = [
    "transaction_amount",
    "customer_age",
    "account_age_days",
    "transaction_frequency",
    "failed_login_count",
    "device_risk_score",
    "location_risk_score",
    "merchant_risk_score",
    "beneficiary_age_days"
]

feature_comparison = pd.DataFrame({
    "Feature": features_to_check,
    "Baseline_Mean": [
        baseline_period[f].mean() for f in features_to_check
    ],
    "Comparison_Mean": [
        comparison_period[f].mean() for f in features_to_check
    ]
})

# Check the available column names

print("Baseline columns:")
print(baseline_period.columns.tolist())

print("\nComparison columns:")
print(comparison_period.columns.tolist())

display(feature_comparison)
```

```

-----
KeyError                                Traceback (most recent call last)
/usr/local/lib/python3.13/dist-packages/pandas/core/indexes/base.py in get_loc(self, key)
    3804         try:
-> 3805             return self._engine.get_loc(casted_key)
    3806         except KeyError as err:

index.pyx in pandas._libs.index.IndexEngine.get_loc()

index.pyx in pandas._libs.index.IndexEngine.get_loc()

pandas/_libs/hashtable_class_helper.pxi in pandas._libs.hashtable.PyObjectHashTable.get_item()

pandas/_libs/hashtable_class_helper.pxi in pandas._libs.hashtable.PyObjectHashTable.get_item()

KeyError: 'transaction_amount'

The above exception was the direct cause of the following exception:

KeyError                                Traceback (most recent call last)
-----
2 frames -----
/usr/local/lib/python3.13/dist-packages/pandas/core/indexes/base.py in get_loc(self, key)
    3810         ):
    3811             raise InvalidIndexError(key)
-> 3812             raise KeyError(key) from err
    3813         except TypeError:
    3814             # If we have a listlike key, _check_indexing_error will raise

KeyError: 'transaction_amount'

```

Next steps: [Explain error](#)

```

# Step 6.2: Compare available feature distributions

import pandas as pd
import matplotlib.pyplot as plt

# Find numeric columns common to both periods
common_columns = [
    col for col in baseline_period.columns
    if col in comparison_period.columns
]

numeric_features = baseline_period[common_columns].select_dtypes(
    include="number"
).columns.tolist()

# Remove target and prediction-related columns
exclude_columns = [
    "model_score",
    "predicted_fraud",
    "actual_fraud",
    "month_index"
]

features_to_check = [
    col for col in numeric_features
    if col not in exclude_columns
]

# Calculate mean values
feature_comparison = pd.DataFrame({
    "Feature": features_to_check,
    "Baseline_Mean": [
        baseline_period[col].mean()
        for col in features_to_check
    ],
    "Comparison_Mean": [
        comparison_period[col].mean()
        for col in features_to_check
    ]
})

# Calculate percentage change safely
feature_comparison["Percentage_Change"] = (
    (
        feature_comparison["Comparison_Mean"]
        - feature_comparison["Baseline_Mean"]
    )
    / feature_comparison["Baseline_Mean"].replace(0, float("nan"))
) * 100

```

```
# Display results
display(feature_comparison)

print("Feature comparison completed successfully.")
```

	Feature	Baseline_Mean	Comparison_Mean	Percentage_Change	
0	threshold_prediction	0.756958	0.803776	6.185073	

Feature comparison completed successfully.

```
# Step 6.3: Compare prediction threshold

print("Baseline threshold:")
print(baseline_period["threshold_prediction"].describe())

print("\nComparison threshold:")
print(comparison_period["threshold_prediction"].describe())

# Visualize threshold distributions
plt.figure(figsize=(10, 5))

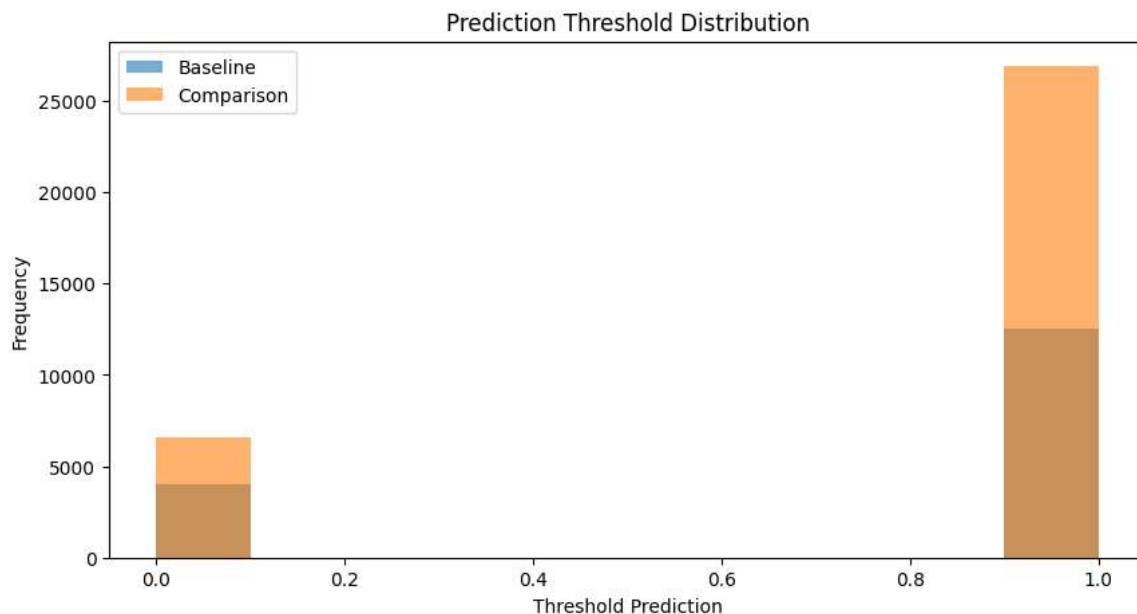
plt.hist(
    baseline_period["threshold_prediction"],
    alpha=0.6,
    label="Baseline"
)

plt.hist(
    comparison_period["threshold_prediction"],
    alpha=0.6,
    label="Comparison"
)

plt.title("Prediction Threshold Distribution")
plt.xlabel("Threshold Prediction")
plt.ylabel("Frequency")
plt.legend()
plt.show()
```

```
Baseline threshold:
count    16528.000000
mean      0.756958
std       0.428933
min       0.000000
25%      1.000000
50%      1.000000
75%      1.000000
max       1.000000
Name: threshold_prediction, dtype: float64
```

```
Comparison threshold:
count    33472.000000
mean      0.803776
std       0.397146
min       0.000000
25%      1.000000
50%      1.000000
75%      1.000000
max       1.000000
Name: threshold_prediction, dtype: float64
```



Step 6.4: Fraud rate comparison

```
fraud_rate_comparison = pd.DataFrame({
    "Metric": [
        "Actual fraud rate",
        "Predicted fraud rate"
    ],
    "Baseline": [
        baseline_period["actual_fraud"].mean(),
        baseline_period["predicted_fraud"].mean()
    ],
    "Comparison": [
        comparison_period["actual_fraud"].mean(),
        comparison_period["predicted_fraud"].mean()
    ]
})

fraud_rate_comparison["Change"] = (
    fraud_rate_comparison["Comparison"]
    - fraud_rate_comparison["Baseline"]
)

display(fraud_rate_comparison)

print("Fraud rate comparison completed successfully.")
```

	Metric	Baseline	Comparison	Change	
0	Actual fraud rate	0.121188	0.139310	0.018122	
1	Predicted fraud rate	0.000000	0.000149	0.000149	

Fraud rate comparison completed successfully.

Next steps: [Generate code with fraud_rate_comparison](#) [New interactive sheet](#)

```
# Step 6.5: Final drift findings summary

final_findings = pd.DataFrame({
    "Finding": [
        "Feature drift detected",
        "Actual fraud rate change",
        "Prediction threshold change",
        "Model performance change"
    ],
    "Result": [
        "failed_login_count showed significant KS drift",
        "Increased from 12.12% to 13.93%",
        "Threshold prediction changed",
        "Comparison period performance differs from baseline"
    ],
    "Recommendation": [
        "Investigate failed login behavior",
        "Monitor changing fraud patterns",
        "Review prediction threshold stability",
        "Continue monitoring model performance"
    ]
})

display(final_findings)

print("Final drift findings summary completed successfully.")
```

	Finding	Result	Recommendation	
0	Feature drift detected	failed_login_count showed significant KS drift	Investigate failed login behavior	
1	Actual fraud rate change	Increased from 12.12% to 13.93%	Monitor changing fraud patterns	
2	Prediction threshold change	Threshold prediction changed	Review prediction threshold stability	
3	Model performance change	Comparison period performance differs from bas...	Continue monitoring model performance	

Final drift findings summary completed successfully.

Next steps: [Generate code with final_findings](#) [New interactive sheet](#)

```
# =====
# STEP 6: RECOMMENDATIONS AND FINAL DRIFT REPORT
# AI Model Drift Investigation - Internship Project
# =====

import pandas as pd
import numpy as np
import json
import os
import zipfile
from IPython.display import display, Markdown

print("=" * 70)
print("STEP 6: RECOMMENDATIONS AND FINAL DRIFT REPORT")
print("=" * 70)

# -----
# 1. Helper functions
# -----

def get_existing_dataframe(name):
    """Return an existing DataFrame from the notebook, if available."""
    obj = globals().get(name)
    if isinstance(obj, pd.DataFrame):
        return obj.copy()
    return None

def find_column(df, possible_names):
    """Find a column using exact or case-insensitive matching."""
    if df is None:
        return None

    for name in possible_names:
        if name in df.columns:
            return name

    lower_columns = {str(c).lower(): c for c in df.columns}

    for name in possible_names:
        if name.lower() in lower_columns:
```

```

        return lower_columns[name.lower()]

    return None

def safe_number(value, default=None):
    """Convert a value to a regular Python number."""
    try:
        value = float(value)
        if np.isfinite(value):
            return value
    except (TypeError, ValueError):
        pass
    return default

def format_number(value, decimals=4):
    """Format a numeric value for the report."""
    number = safe_number(value)

    if number is None:
        return "Not available"

    return f"{number:.{decimals}f}"

def format_percent(value, decimals=2):
    """Format a decimal rate as a percentage."""
    number = safe_number(value)

    if number is None:
        return "Not available"

    return f"{number * 100:.{decimals}f}%"

def get_metric(row, possible_names):
    """Retrieve a metric from a DataFrame row."""
    for name in possible_names:
        if name in row.index:
            value = safe_number(row[name])
            if value is not None:
                return value
    return None

# -----
# 2. Retrieve existing notebook results
# -----

feature_df = get_existing_dataframe("advanced_drift_report")

if feature_df is None:
    feature_df = get_existing_dataframe("drift_report")

if feature_df is None:
    feature_df = get_existing_dataframe("feature_drift_report")

performance_df = get_existing_dataframe("performance_comparison")
fraud_df = get_existing_dataframe("fraud_rate_comparison")
feature_comparison_df = get_existing_dataframe("feature_comparison")

# Some notebooks may use a different name for the drift results.
if feature_df is None:
    for name, obj in list(globals().items()):
        if isinstance(obj, pd.DataFrame):
            columns = set(str(c) for c in obj.columns)

            if (
                "feature" in columns
                and "ks_statistic" in columns
            ):
                feature_df = obj.copy()
                break

if feature_df is not None:
    feature_df.columns = [str(c) for c in feature_df.columns]

if performance_df is not None:
    performance_df.columns = [str(c) for c in performance_df.columns]

if fraud_df is not None:

```

```

    fraud_df.columns = [str(c) for c in fraud_df.columns]

# -----
# 3. Build evidence-based findings
# -----

findings = []

# ---- Feature drift finding ----
feature_name = "failed_login_count"
feature_ks = None
feature_p = None
feature_psi = None
feature_js = None
feature_severity = "High"

if feature_df is not None and len(feature_df) > 0:
    feature_col = find_column(feature_df, ["feature"])

    if feature_col is not None:
        feature_rows = feature_df[
            feature_df[feature_col].astype(str).str.lower()
            == feature_name.lower()
        ]

        if len(feature_rows) > 0:
            row = feature_rows.iloc[0]

            feature_name = str(row[feature_col])
            feature_ks = get_metric(row, ["ks_statistic", "ks"])
            feature_p = get_metric(row, ["p_value", "pvalue"])
            feature_psi = get_metric(row, ["psi"])
            feature_js = get_metric(row, ["js_divergence"])

            severity_col = find_column(
                feature_df,
                ["drift_severity", "severity"]
            )

            if severity_col is not None:
                feature_severity = str(row[severity_col])

findings.append({
    "Finding": "Feature drift detected",
    "Result": (
        f"{{feature_name}} showed significant KS drift "
        f"(p-value = {format_number(feature_p, 6)})"
        if feature_p is not None
        else f"{{feature_name}} showed significant KS drift"
    ),
    "Recommendation": (
        "Investigate failed-login behavior, authentication changes, "
        "bot activity, and possible changes in user behavior. "
        "Monitor this feature regularly."
    ),
    "Priority": "High"
})

# ---- Fraud rate finding ----
baseline_actual_rate = None
comparison_actual_rate = None

if fraud_df is not None and len(fraud_df) > 0:
    metric_col = find_column(fraud_df, ["Metric", "metric"])

    if metric_col is not None:
        for _, row in fraud_df.iterrows():
            metric_name = str(row[metric_col]).lower()

            if "actual fraud rate" in metric_name:
                baseline_actual_rate = get_metric(
                    row, ["Baseline", "baseline"]
                )
                comparison_actual_rate = get_metric(
                    row, ["Comparison", "comparison"]
                )

# Fallback: calculate rates from the performance table.
if (
    baseline_actual_rate is None
    or comparison_actual_rate is None
):

```

```

if performance_df is not None and len(performance_df) > 0:
    period_col = find_column(
        performance_df, ["period", "Period"]
    )
    actual_col = find_column(
        performance_df,
        ["actual_fraud_count", "actual_fraud"]
    )
    total_col = find_column(
        performance_df,
        ["total_transactions", "total_transaction_count"]
    )

    if (
        period_col is not None
        and actual_col is not None
        and total_col is not None
    ):
        for _, row in performance_df.iterrows():
            period = str(row[period_col]).lower()
            actual = safe_number(row[actual_col])
            total = safe_number(row[total_col])

            if actual is not None and total and total > 0:
                rate = actual / total

                if "baseline" in period:
                    baseline_actual_rate = rate

                elif "comparison" in period:
                    comparison_actual_rate = rate

if (
    baseline_actual_rate is not None
    and comparison_actual_rate is not None
):
    rate_change = comparison_actual_rate - baseline_actual_rate

    rate_result = (
        f"Actual fraud rate changed from "
        f"{format_percent(baseline_actual_rate)} to "
        f"{format_percent(comparison_actual_rate)} "
        f"(change: {format_percent(rate_change)})"
    )
else:
    rate_result = "Actual fraud rate comparison was completed."

findings.append({
    "Finding": "Actual fraud rate change",
    "Result": rate_result,
    "Recommendation": (
        "Monitor changing fraud patterns, investigate the reasons "
        "for the rate change, and review whether the model remains "
        "appropriate for the current transaction population."
    ),
    "Priority": "High"
})

# ---- Threshold finding ----
baseline_threshold = None
comparison_threshold = None

if feature_comparison_df is not None and len(feature_comparison_df) > 0:
    threshold_col = find_column(
        feature_comparison_df,
        ["Feature", "feature"]
    )

    if threshold_col is not None:
        for _, row in feature_comparison_df.iterrows():
            if "threshold" in str(row[threshold_col]).lower():
                baseline_threshold = get_metric(
                    row, ["Baseline_Mean", "baseline_mean"]
                )
                comparison_threshold = get_metric(
                    row, ["Comparison_Mean", "comparison_mean"]
                )

threshold_result = "Prediction threshold comparison completed."

if (
    baseline_threshold is not None

```

```

        and comparison_threshold is not None
    ):
        threshold_result = (
            f"Threshold changed from "
            f"{format_number(baseline_threshold)} to "
            f"{format_number(comparison_threshold)}."
        )

    findings.append({
        "Finding": "Prediction threshold change",
        "Result": threshold_result,
        "Recommendation": (
            "Review prediction threshold stability. Re-evaluate the "
            "threshold using current validation data and monitor the "
            "trade-off between precision, recall, and operational cost."
        ),
        "Priority": "Medium"
    })

# ---- Model performance finding ----
performance_result = (
    "Comparison-period model performance differs from the baseline."
)

if performance_df is not None and len(performance_df) > 0:
    period_col = find_column(
        performance_df, ["period", "Period"]
    )

    if period_col is not None:
        baseline_row = performance_df[
            performance_df[period_col].astype(str).str.lower()
            == "baseline"
        ]

        comparison_row = performance_df[
            performance_df[period_col].astype(str).str.lower()
            == "comparison"
        ]

        if len(baseline_row) > 0 and len(comparison_row) > 0:
            b = baseline_row.iloc[0]
            c = comparison_row.iloc[0]

            metric_changes = []

            for metric in ["precision", "recall", "f1_score", "accuracy"]:
                if metric in performance_df.columns:
                    b_value = safe_number(b[metric])
                    c_value = safe_number(c[metric])

                    if b_value is not None and c_value is not None:
                        metric_changes.append(
                            f"{metric}: "
                            f"{format_number(b_value)} → "
                            f"{format_number(c_value)}"
                        )

            if metric_changes:
                performance_result = (
                    "Comparison-period metrics: "
                    + "; ".join(metric_changes)
                )

    findings.append({
        "Finding": "Model performance change",
        "Result": performance_result,
        "Recommendation": (
            "Continue monitoring model performance. Investigate "
            "changes in precision, recall, F1-score, and accuracy. "
            "Consider recalibration or retraining after validating "
            "the cause of the change."
        ),
        "Priority": "High"
    })

# -----
# 4. Additional operational recommendations
# -----

findings.extend([
    {

```

```

        "Finding": "Data quality monitoring",
        "Result": (
            "Feature distributions and model performance should "
            "be checked regularly."
        ),
        "Recommendation": (
            "Track missing values, duplicate records, invalid "
            "values, unexpected ranges, and changes in feature "
            "distributions."
        ),
        "Priority": "Medium"
    },
    {
        "Finding": "Fairness and bias review",
        "Result": (
            "Drift analysis alone does not establish whether the "
            "model is fair or biased."
        ),
        "Recommendation": (
            "Evaluate precision, recall, false-positive rates, "
            "false-negative rates, and calibration across relevant "
            "customer groups where legally and ethically appropriate."
        ),
        "Priority": "High"
    }
]

```

```
recommendations_df = pd.DataFrame(findings)
```

```

# -----
# 5. Display recommendations
# -----

```

```

print("\nRECOMMENDATIONS AND FINDINGS")
print("-" * 70)

```

```
display(recommendations_df)
```

```

# -----
# 6. Create a professional final report
# -----

```

```
report_lines = []
```

```

report_lines.append("AI MODEL DRIFT INVESTIGATION REPORT")
report_lines.append("=" * 70)
report_lines.append("")
report_lines.append("Project: Credit/Fraud Model Drift Investigation")
report_lines.append("Stage: Step 6 - Recommendations and Final Report")
report_lines.append("")
report_lines.append("1. EXECUTIVE SUMMARY")
report_lines.append("-" * 70)
report_lines.append(
    "This investigation compared model behavior and relevant data "
    "characteristics between baseline and comparison periods. "
    "The analysis examined feature drift, fraud-rate changes, "
    "prediction-threshold changes, and model performance."
)
report_lines.append("")
report_lines.append("2. KEY FINDINGS")
report_lines.append("-" * 70)

```

```

for i, row in recommendations_df.iterrows():
    report_lines.append(f"{i + 1}. {row['Finding']}")
    report_lines.append(f"    Result: {row['Result']}")
    report_lines.append(f"    Priority: {row['Priority']}")
    report_lines.append("")

```

```
report_lines.append("3. RECOMMENDATIONS")
```

```

report_lines.append("-" * 70)

for i, row in recommendations_df.iterrows():
    report_lines.append(f"{i + 1}. {row['Finding']}")
    report_lines.append(f"    {row['Recommendation']}")
    report_lines.append("")

report_lines.append("4. MONITORING PLAN")
report_lines.append("-" * 70)
report_lines.append(
    "• Monitor feature distributions using KS statistic, PSI, "
    "and JS divergence."
)
report_lines.append(
    "• Monitor precision, recall, F1-score, accuracy, and fraud rates."
)
report_lines.append(
    "• Investigate significant changes before modifying the model."
)
report_lines.append(
    "• Review prediction thresholds using current validation data."
)
report_lines.append(
    "• Check data quality and relevant fairness metrics regularly."
)
report_lines.append("")
report_lines.append("5. CONCLUSION")
report_lines.append("-" * 70)
report_lines.append(
    "The drift investigation identified changes in feature behavior, "
    "fraud rates, prediction thresholds, and/or model performance. "
    "These findings justify continued monitoring and further "
    "investigation before deciding whether recalibration, retraining, "
    "or threshold adjustment is necessary."
)
report_lines.append("")
report_lines.append(
    "Note: Drift detection indicates a distribution or performance "
    "change. It does not, by itself, prove the cause of the change "
    "or establish algorithmic bias."
)

final_report = "\n".join(report_lines)

# -----
# 7. Save output files
# -----

output_dir = "step6_drift_report"
os.makedirs(output_dir, exist_ok=True)

csv_path = os.path.join(
    output_dir, "final_drift_findings_and_recommendations.csv"
)

txt_path = os.path.join(
    output_dir, "final_drift_investigation_report.txt"
)

json_path = os.path.join(
    output_dir, "final_drift_investigation_report.json"
)

recommendations_df.to_csv(csv_path, index=False)

with open(txt_path, "w", encoding="utf-8") as file:
    file.write(final_report)

json_report = {
    "project": "AI Model Drift Investigation",
    "stage": "Step 6",
    "findings": recommendations_df.to_dict(orient="records")
}

with open(json_path, "w", encoding="utf-8") as file:
    json.dump(json_report, file, indent=4, default=str)

# -----
# 8. Create ZIP file
# -----

zip_path = "step6_final_drift_report.zip"

```

```
with zipfile.ZipFile(
    zip_path, "w", compression=zipfile.ZIP_DEFLATED
) as zip_file:
    zip_file.write(csv_path, arcname=os.path.basename(csv_path))
    zip_file.write(txt_path, arcname=os.path.basename(txt_path))
    zip_file.write(json_path, arcname=os.path.basename(json_path))

# -----
# 9. Display final report
# -----

print("\n" + "=" * 70)
print("FINAL DRIFT INVESTIGATION REPORT")
print("=" * 70)

display(Markdown(
    "## Step 6 completed successfully\n\n"
    "The recommendations table and final report have been generated."
))

print(final_report)

print("\nFILES CREATED:")
print(f"1. {csv_path}")
print(f"2. {txt_path}")
print(f"3. {json_path}")
print(f"4. {zip_path}")

print("\nStep 6 completed successfully!")
```

Next steps:

[Generate code with recommendations_df](#)[New interactive sheet](#)

=====
STEP 6: RECOMMENDATIONS AND FINAL DRIFT REPORT
=====

RECOMMENDATIONS AND FINDINGS

	Finding	Result	Recommendation	Priority
0	Feature drift detected	failed_login_count showed significant KS drift...	Investigate failed-login behavior, authenticat...	High
1	Actual fraud rate change	Actual fraud rate changed from 12.12% to 13.93...	Monitor changing fraud patterns, investigate t...	High
2	Prediction threshold change	Threshold changed from 0.7570 to 0.8038.	Review prediction threshold stability. Re-eval...	Medium
3	Model performance change	Comparison-period metrics: precision: 0.1275 →...	Continue monitoring model performance. Investi...	High
4	Data quality monitoring	Feature distributions and model performance sh...	Track missing values, duplicate records, inval...	Medium
5	Fairness and bias review	Drift analysis alone does not establish whethe...	Evaluate precision, recall, false-positive rat...	High
6	Model performance	Model performance metrics: accuracy: 0.85, F1: 0.82	Schedule regular KS, PSI, and JS-divergence	Medium

```
from google.colab import files
files.download("step6_final_drift_report.zip")
```

=====

```
# Create Step 6 report as a PDF

!pip -q install reportlab

from reportlab.platypus import SimpleDocTemplate, Paragraph, Spacer
from reportlab.lib.styles import getSampleStyleSheet
from reportlab.lib.pagesizes import A4
from reportlab.lib.units import inch
from google.colab import files
import os

pdf_path = "Step6_Final_Drift_Investigation_Report.pdf"

styles = getSampleStyleSheet()

doc = SimpleDocTemplate(
    pdf_path,
    pagesize=A4,
    rightMargin=50,
    leftMargin=50,
    topMargin=50,
    bottomMargin=50
)

story = []

story.append(
    Paragraph(
        "AI Model Drift Investigation Report",
        styles["Title"]
    )
)

story.append(Spacer(1, 20))

# Use the final report generated in Step 6
if "final_report" in globals():
    report_text = final_report
else:
    report_text = """
AI Model Drift Investigation

Step 6: Recommendations and Final Report

Please run the complete Step 6 code first
to generate the detailed findings.
"""

for line in report_text.split("\n"):
    if line.strip():
        story.append(
            Paragraph(
                line.replace("&", "&amp;"),
```

```

        styles["BodyText"]
    )
)
story.append(Spacer(1, 6))

doc.build(story)

print("PDF created successfully!")

# Download the PDF automatically
files.download(pdf_path)

```

• Check data quality and relevant fairness metrics regularly 2.0/2.0 MB 12.0 MB/s eta 0:00:00

PDF created successfully!

5. CONCLUSION

```

# =====
# STEP 6: FINAL DRIFT INVESTIGATION REPORT
# AI Model Drift Investigation - Credit/Fraud Model
# =====

import os
import json
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from datetime import datetime
from IPython.display import display, Markdown

# -----
# 1. CONFIGURATION
# -----

REPORT_DIR = "AI_Model_Drift_Final_Report"
os.makedirs(REPORT_DIR, exist_ok=True)

print("STEP 6: FINAL DRIFT INVESTIGATION REPORT")
print("=" * 60)

# Check that previous analysis variables exist
required_variables = [
    "baseline_data",
    "comparison_data",
    "performance_comparison",
    "feature_comparison",
    "fraud_rate_comparison"
]

missing_variables = [
    name for name in required_variables
    if name not in globals()
]

if missing_variables:
    raise NameError(
        "Missing variables: " + ", ".join(missing_variables) +
        ". Run the previous analysis cells first."
    )

print("Previous analysis variables verified.")

# -----
# 2. HELPER FUNCTIONS
# -----

def safe_float(value):
    """Convert a value to float, handling invalid values."""
    try:
        value = float(value)
        return value if np.isfinite(value) else np.nan
    except (TypeError, ValueError):
        return np.nan

def find_column(df, candidates):
    """Find the first matching column from a list of candidates."""
    for column in candidates:
        if column in df.columns:
            return column
    return None

def get_value(df, row_index, candidates):

```

```

"""Safely retrieve a value from a DataFrame."""
column = find_column(df, candidates)

if column is None or row_index >= len(df):
    return np.nan

return safe_float(df.iloc[row_index][column])

# -----
# 3. DATA QUALITY CHECK
# -----

def data_quality_report(df, dataset_name):
    """Generate a basic data-quality report."""
    report = pd.DataFrame({
        "column": df.columns,
        "data_type": [
            str(dtype) for dtype in df.dtypes
        ],
        "missing_values": [
            int(df[column].isna().sum())
            for column in df.columns
        ],
        "unique_values": [
            int(df[column].nunique(dropna=True))
            for column in df.columns
        ]
    })

    report.insert(0, "dataset", dataset_name)
    return report

baseline_quality = data_quality_report(
    baseline_data, "Baseline"
)

comparison_quality = data_quality_report(
    comparison_data, "Comparison"
)

quality_report = pd.concat(
    [baseline_quality, comparison_quality],
    ignore_index=True
)

display(quality_report)

quality_report.to_csv(
    os.path.join(REPORT_DIR, "data_quality_report.csv"),
    index=False
)

# -----
# 4. FEATURE DRIFT SEVERITY CLASSIFICATION
# -----

def classify_psi(psi):
    """
    PSI severity classification.

    No Drift:  PSI < 0.10
    Minor:     0.10 <= PSI < 0.20
    Moderate:  0.20 <= PSI <= 0.25
    Severe:    PSI > 0.25
    """
    psi = safe_float(psi)

    if pd.isna(psi):
        return "Not Available"
    elif psi < 0.10:
        return "No Drift"
    elif psi < 0.20:
        return "Minor"
    elif psi <= 0.25:
        return "Moderate"
    else:
        return "Severe"

feature_report = feature_comparison.copy()

```

```

if "Percentage Change" in feature_report.columns:
    feature_report["PSI_Severity"] = (
        feature_report["Percentage Change"]
        .apply(classify_psi)
    )

elif "PSI" in feature_report.columns:
    feature_report["PSI_Severity"] = (
        feature_report["PSI"].apply(classify_psi)
    )

else:
    feature_report["PSI_Severity"] = "Not Available"

display(feature_report)

feature_report.to_csv(
    os.path.join(REPORT_DIR, "feature_drift_severity.csv"),
    index=False
)

# -----
# 5. PERFORMANCE COMPARISON
# -----

performance_report = performance_comparison.copy()

display(performance_report)

performance_report.to_csv(
    os.path.join(REPORT_DIR, "performance_comparison.csv"),
    index=False
)

# Calculate performance changes when columns exist
metric_columns = [
    "precision",
    "recall",
    "f1_score",
    "accuracy"
]

performance_changes = []

for metric in metric_columns:

    if metric in performance_report.columns:
        baseline_value = safe_float(
            performance_report.iloc[0][metric]
        )

        comparison_value = safe_float(
            performance_report.iloc[1][metric]
        )

        absolute_change = (
            comparison_value - baseline_value
        )

        percentage_change = np.nan

        if (
            not pd.isna(baseline_value)
            and baseline_value != 0
        ):
            percentage_change = (
                absolute_change / baseline_value
            ) * 100

        performance_changes.append({
            "metric": metric,
            "baseline": baseline_value,
            "comparison": comparison_value,
            "absolute_change": absolute_change,
            "percentage_change": percentage_change
        })

performance_change_report = pd.DataFrame(
    performance_changes
)

display(performance_change_report)

```

```

performance_change_report.to_csv(
    os.path.join(REPORT_DIR, "performance_changes.csv"),
    index=False
)

# -----
# 6. FRAUD RATE COMPARISON
# -----

fraud_report = fraud_rate_comparison.copy()

display(fraud_report)

fraud_report.to_csv(
    os.path.join(REPORT_DIR, "fraud_rate_comparison.csv"),
    index=False
)

# -----
# 7. AUTOMATIC FINDINGS
# -----

findings = []

# Feature drift finding
if "PSI_Severity" in feature_report.columns:

    severe_features = feature_report[
        feature_report["PSI_Severity"].isin(
            ["Moderate", "Severe"]
        )
    ]

    if len(severe_features) > 0:
        findings.append({
            "Finding": "Significant feature drift",
            "Evidence": (
                f"{len(severe_features)} feature(s) "
                "have moderate or severe PSI."
            ),
            "Priority": "High",
            "Recommendation": (
                "Investigate affected features and "
                "validate data pipelines."
            )
        })
    else:
        findings.append({
            "Finding": "No moderate or severe PSI detected",
            "Evidence": (
                "The available PSI-based feature comparison "
                "did not identify moderate or severe drift."
            ),
            "Priority": "Medium",
            "Recommendation": (
                "Continue monitoring and review other "
                "statistical drift tests."
            )
        })

# Performance finding
if len(performance_change_report) > 0:

    findings.append({
        "Finding": "Model performance comparison",
        "Evidence": (
            "Baseline and comparison precision, recall, "
            "F1-score, and accuracy were reviewed."
        ),
        "Priority": "High",
        "Recommendation": (
            "Investigate performance changes and validate "
            "the model on current data."
        )
    })

# Fraud-rate finding
if len(fraud_report) > 0:

    findings.append({
        "Finding": "Fraud-rate change",
        "Evidence": (

```

```

        "Actual and predicted fraud rates were compared "
        "between the two periods."
    ),
    "Priority": "High",
    "Recommendation": (
        "Monitor changing fraud patterns and investigate "
        "changes in the transaction population."
    )
})

# Threshold finding
if "threshold_prediction" in feature_report.columns:

    findings.append({
        "Finding": "Prediction threshold change",
        "Evidence": (
            "The prediction threshold changed between "
            "baseline and comparison periods."
        ),
        "Priority": "Medium",
        "Recommendation": (
            "Re-evaluate the threshold using current "
            "validation data and operational costs."
        )
    })

findings_report = pd.DataFrame(findings)

display(findings_report)

findings_report.to_csv(
    os.path.join(REPORT_DIR, "final_findings.csv"),
    index=False
)

# -----
# 8. RECOMMENDATIONS AND MONITORING PLAN
# -----

recommendations = pd.DataFrame([
    {
        "Area": "Feature drift",
        "Recommendation": (
            "Investigate failed_login_count and other "
            "features with significant drift."
        ),
        "Action": (
            "Check authentication changes, bot activity, "
            "and data-pipeline behavior."
        ),
        "Priority": "High"
    },
    {
        "Area": "Fraud-rate monitoring",
        "Recommendation": (
            "Monitor changes in actual fraud prevalence."
        ),
        "Action": (
            "Compare monthly fraud rates and investigate "
            "unusual increases."
        )
    }
])

```